
Author(s)	David Feuerstein
Restrictions	Customer confidential - Vector decides
Abstract	This application note describes how access to ENDINIT protected registers on Infineon TriCore derivatives should be handled. This application note describes how access to ENDINIT protected registers on Infineon TriCore derivatives should be handled..

Table of Contents

1.0	Overview	2
1.1	ENDINIT Mechanism.....	2
1.2	Modification of ENDINIT Bit.....	2
1.3	Effects on Watchdog Timer	2
1.4	Multicore	2
2.0	Handling for Vector modules	2
2.1	Vector BSW	2
2.2	Application Code	2
2.3	ENDINIT API	3
2.3.1	Appl_UnlockEndinit.....	3
2.3.2	Appl_LockEndinit	3
2.4	Writing to the ENDINIT Protected Registers with Safe Context OS	4
3.0	Contacts	4

1.0 Overview

This application note describes what has to be done to allow Vector BSW drivers to write to ENDINIT protected registers on Infineon TriCore derivatives.

Note:	Regarding safety project: The correct usage of the ENDINIT access must be assured depending on the target ASIL level by the user.
-------	---

1.1 ENDINIT Mechanism

On Infineon TRICORE Aurix derivatives there is a protection mechanism, which protects registers from unintended writes.

The intention of this mechanism is to avoid modification of registers that are usually only programmed once during the initialization sequence (also see chapter **The Endinit Functions** in TriCore user manual).

But this is not sufficient for AUTOSAR environments, where some registers have to be programmed after the initialization sequence, e.g. to reinitialize the CAN cell after bus off or sleep.

1.2 Modification of ENDINIT Bit

The ENDINIT bit can be unlocked/locked by a special call sequence.

If the ENDINIT bit is unlocked, access on ENDINIT-protected registers is possible again.

Note:	The calls to unlock/lock the ENDINIT bit can be done in user mode.
-------	--

1.3 Effects on Watchdog Timer

If the ENDINIT bit in WDTxCON0 registers is cleared, the watchdog timer will be set into timeout mode automatically. This is to ensure that the ENDINIT bit does not keep unlocked unintentionally.

A special watchdog handling is needed in order to clear ENDINIT while the watchdog is in operation.

Note:	The Infineon MCAL provides APIs to clear ENDINIT with proper watchdog handling (see e.g. <code>Mcal_SetCpuENDINIT</code> and <code>Mcal_ResetCpuENDINIT</code>)
-------	--

1.4 Multicore

In multicore environments the ENDINIT bit is available once per core.

2.0 Handling for Vector modules

2.1 Vector BSW

The Vector BSW drivers like `DrvCan`, `DrvFr`, `DrvLin`, `DrvEth`, `DrvI2c` need to write to their ENDINIT protected registers after the initialization sequence.

These drivers use application callouts to inform the application to unlock/lock the ENDINIT bit (see chapter 2.3).

2.2 Application Code

The implementation of the two callouts must be provided by the application.

The application code has to handle the watchdog timer and has to unlock/lock the ENDINIT bit (see chapter 2.3)

2.3 ENDINIT API

2.3.1 Appl_UnlockEndinit

This function is called by the BSW driver to unlock the ENDINIT bit prior to writing protected configuration registers.

The Watchdog handling for the ENDINIT unlock procedure has to be done here.

This API should be implemented as follows:

- Handle watchdog as recommended via MCAL or TriCore user manual
- Unlock WDTxCON0 via password access
- Clear WDTxCON0.ENDINIT

2.3.2 Appl_LockEndinit

This function is called by the BSW driver to lock the ENDINIT bit after writing protected configuration registers.

The Watchdog handling for the ENDINIT lock procedure has to be done here.

This API should be implemented as follows:

- Handle watchdog as recommended via MCAL or TriCore user manual
- Unlock WDTxCON0 via password access
- Set WDTxCON0.ENDINIT

2.4 Writing to the ENDINIT Protected Registers with Safe Context OS

The registers protected by the ENDINIT mechanism can only be written as supervisor.

Therefore the access to these registers will be handled via `osWritePeripheral32` API inside the Vector BSW drivers. To access those peripheral registers, peripheral regions in OS need to be configured (see Figure 1).

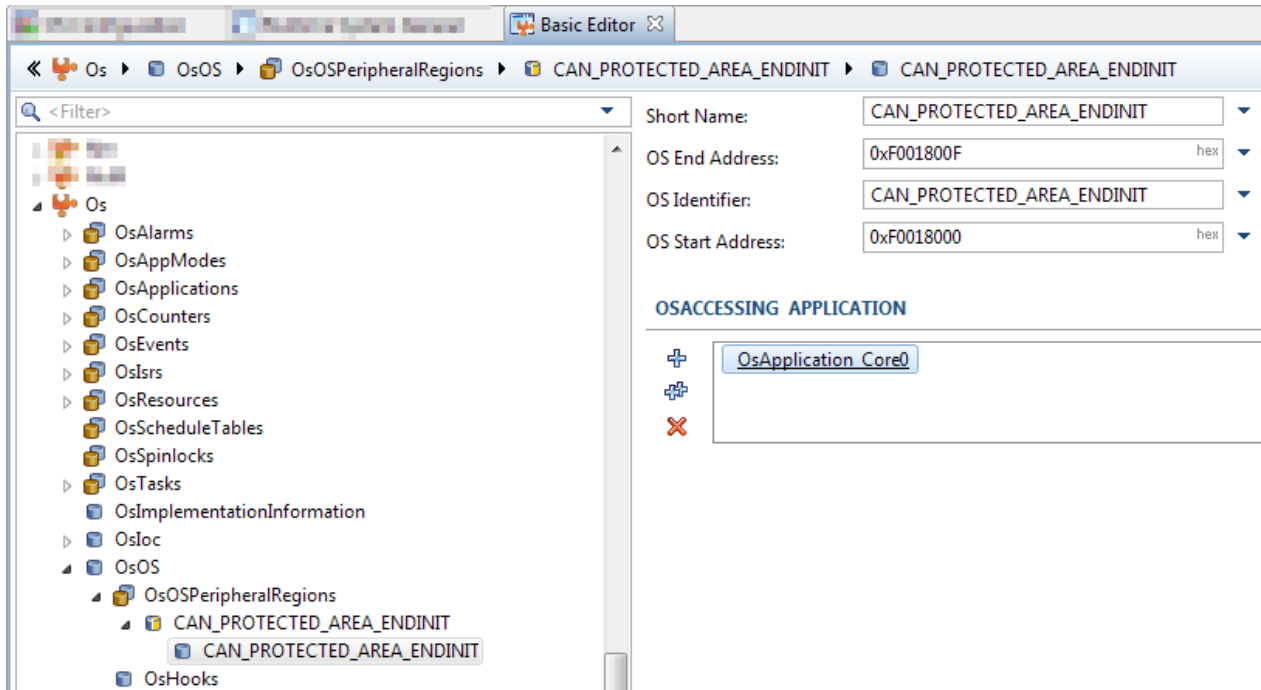


Figure 1 - Configuration of a Peripheral Region in DaVinci Configurator 5

Note: Please refer to the technical reference documentation of the BSW drivers to see which regions have to be configured (names/addresses).

3.0 Contacts

For a full list with all Vector locations and addresses worldwide, please visit <http://vector.com/contact/>.